

Student Record and Academic Management System

By: Nanor Vaibhav R.

- **Problem Statement:**

As we know now, maintaining students record in a school/college is an important as well as a complex task. And when it comes to maintaining the record of 100s of students at the same time that too in a traditional 'Paper based Records' is unreliable, time consuming as well as a stressful job to do. So, a Digital System, that manages all these tasks is needed, so the student's records are reliable, convenient, and easy to access.

- **Functional Requirements:**

1. Add student information
2. Display student records
3. Search student details
4. Calculate average marks
5. Display grades
6. Exit application

- **Non-Functional Requirements:**

1. Simplicity
2. Ease of use
3. Accuracy

4. Reliability

- Algorithms:

1. Adding student records:

- i. Start.
- ii. Ask the user to input the student's name, PRN, department, semester, and attendance.
- iii. Ask the user to input the marks for 5 subjects.
- iv. Calculate the average by calling the Average Calculation algorithm.
- v. Calculate the grade by calling the Grade Calculation algorithm.
- vi. Save all this data as a single student record.
- vii. Add this new record to the main student list.
- viii. End.

2. Searching Records:

- i. Start.
- ii. Check if the student list is empty. If yes, display "No records available" and End.
- iii. Ask the user to input the PRN they want to search for.
- iv. Go through the student list one by one.
- v. If a student's PRN matches the input, display their full details and stop searching.
- vi. If the end of the list is reached and no match is found, display "Student not found".
- vii. End.

3. Grade Calculation:

- i. Start.

- ii. Take the student's average marks as input.
- iii. If average is 90 or above, the grade is "A+".
- iv. If average is 80 to 89, the grade is "A".
- v. If average is 70 to 79, the grade is "B+".
- vi. If average is 60 to 69, the grade is "B".
- vii. If average is 50 to 59, the grade is "C".
- viii. If average is 40 to 49, the grade is "D".
- ix. If average is below 40, the grade is "F".
- x. Return the final grade.
- xi. End.

4. Average Calculation:

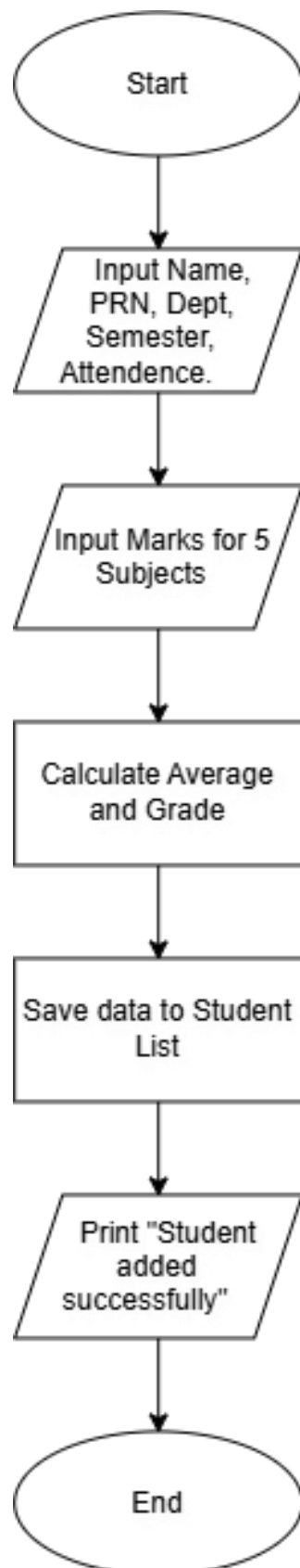
- i. Start.
- ii. Take the marks of the 5 subjects as input.
- iii. Add all 5 marks together to find the total sum.
- iv. Divide the total sum by 5 to find the average.
- v. Return the average.
- vi. End.

5. Displaying Records:

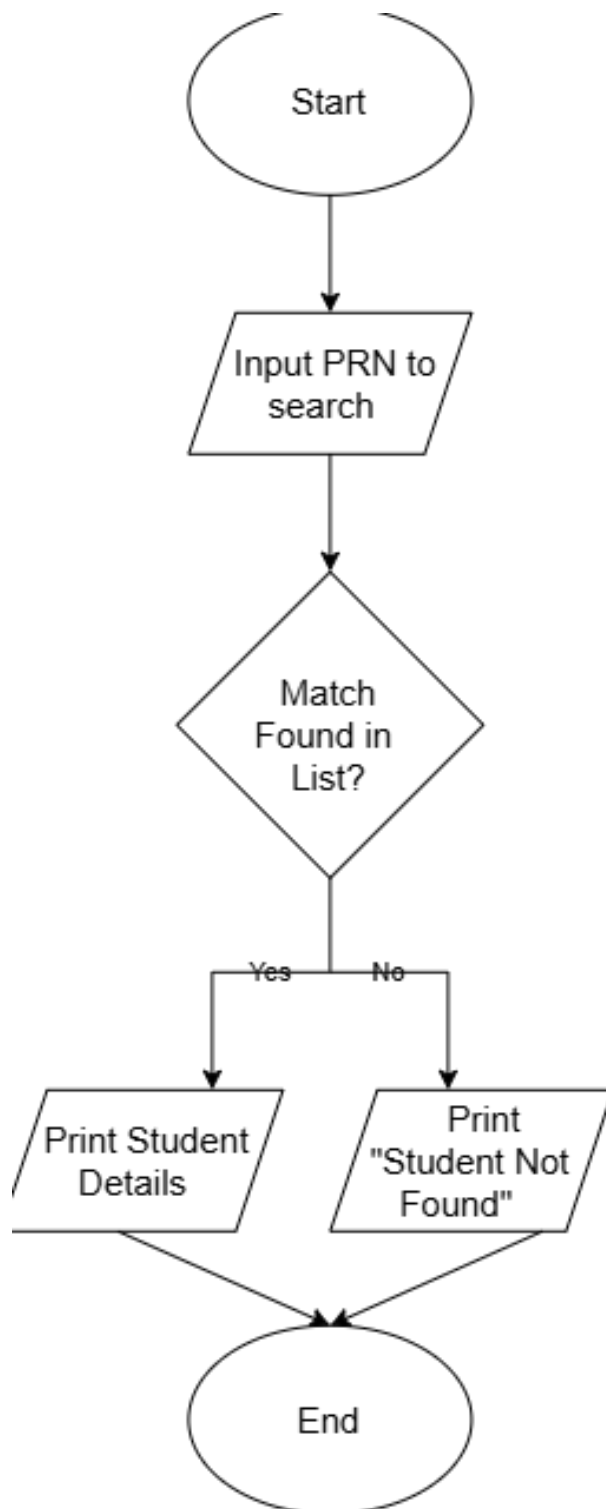
- i. Start.
- ii. Check if the main student list is empty.
- iii. If it is empty, display "No student records available" and End.
- iv. If it is not empty, look at the first student record.
- v. Print that student's name, PRN, department, semester, marks, attendance, average, and grade.
- vi. Move to the next student and repeat step 5 until all students have been displayed.
- vii. End.

- Flowcharts:

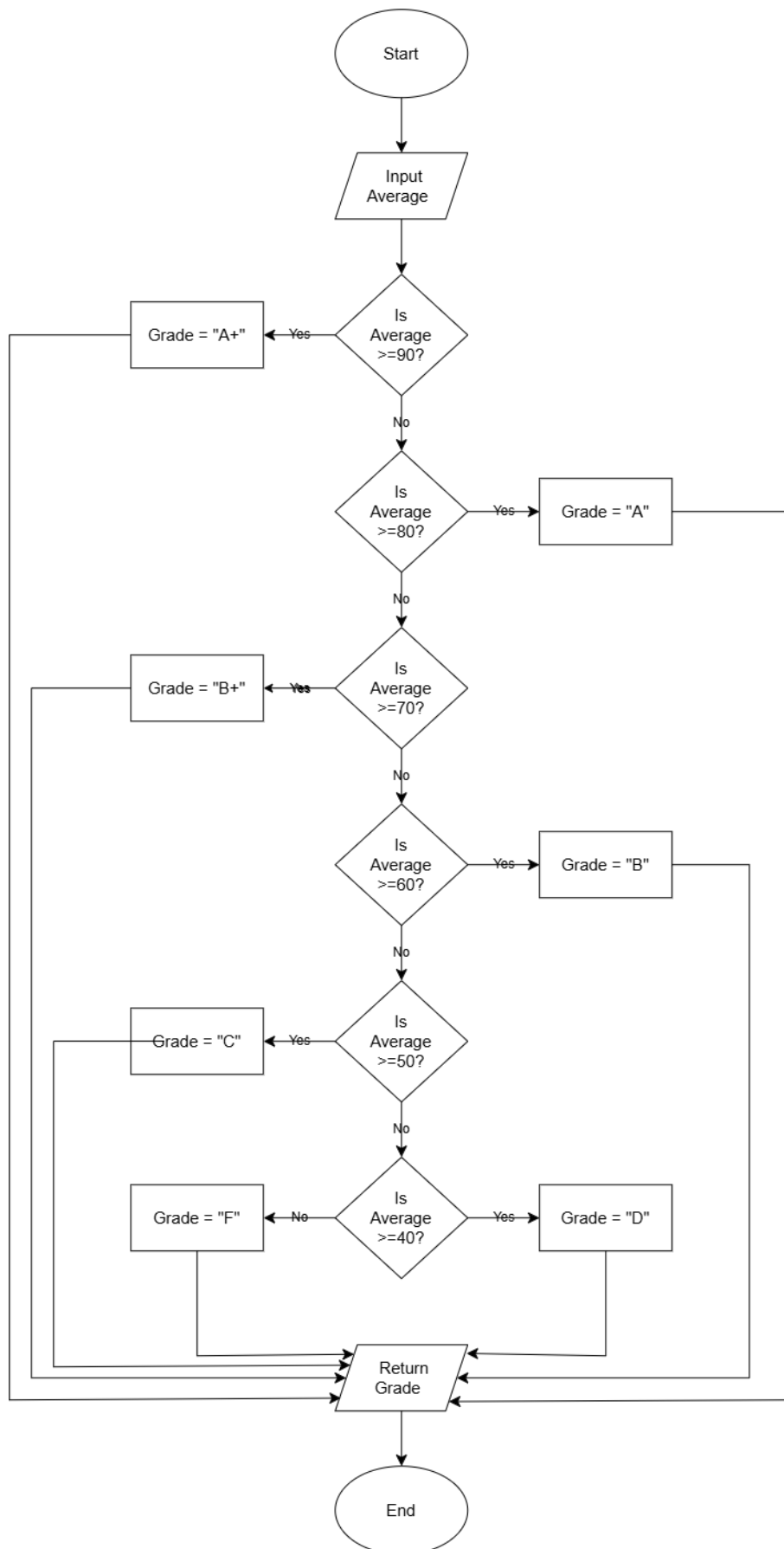
1. Add Student:



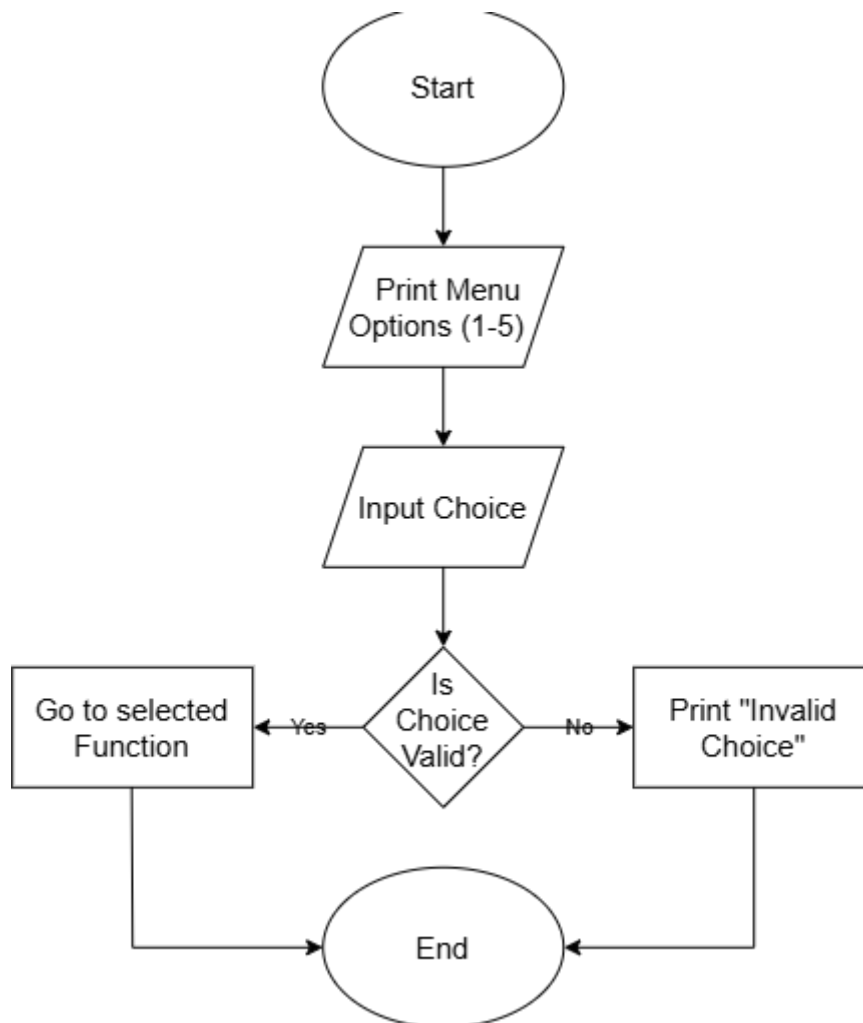
2. Search Student:



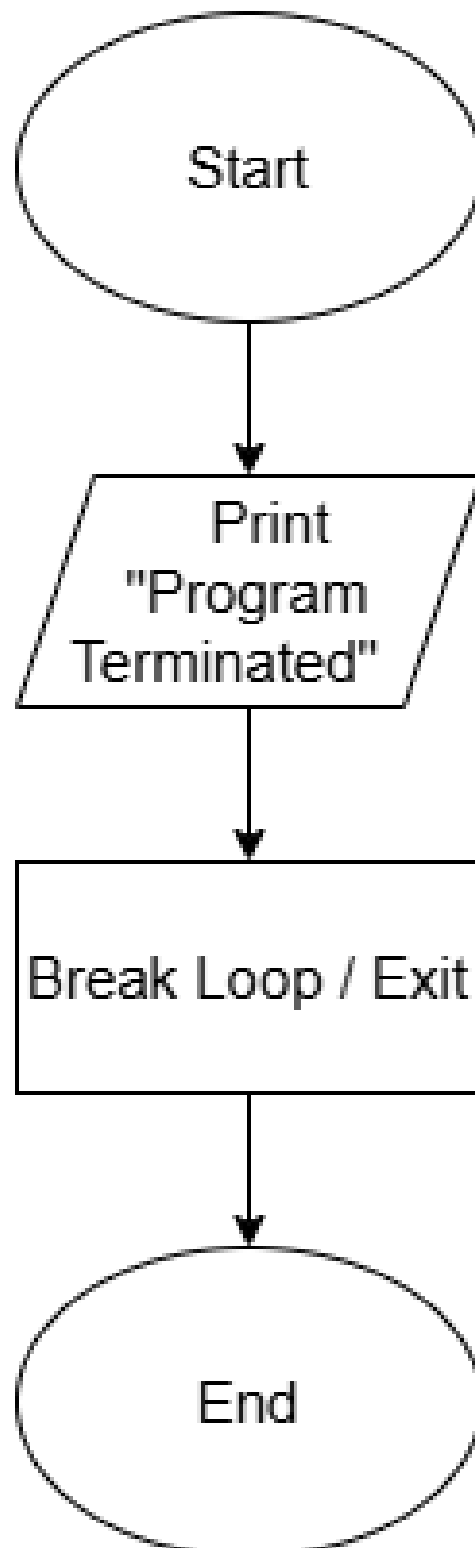
3. Grade Calculation:



4. Display Menu:



5. Exit System



- Pseudocode

1. Pseudocode for Adding a Student:

```
PROCEDURE Add_Student
```

```
    PRINT "Enter student details (name, roll no,  
    department, semester)"
```

```
    INPUT name, roll_no, department, semester
```

```
    INIT marks as empty list
```

```
    FOR i = 1 TO 5
```

```
        INPUT mark
```

```
        APPEND mark TO marks
```

```
    END FOR
```

```
    INPUT attendance
```

```
    average = CALL Calculate_Average(marks)
```

```
    grade = CALL Calculate_Grade(average)
```

```
    CREATE student record with all details
```

```
    ADD student to students list
```

```
    PRINT "Student record added successfully!"
```

```
END PROCEDURE
```

2. Pseudocode for Searching a Student:

```
PROCEDURE Search_Student
  IF students list is empty THEN
    PRINT "No student records available."
    RETURN
  END IF

  INPUT search_roll_no
  SET found = FALSE

  FOR EACH student IN students list
    IF student.roll_no EQUALS search_roll_no THEN
      PRINT student details
      SET found = TRUE
      BREAK
    END IF
  END FOR

  IF found EQUALS FALSE THEN
    PRINT "Student not found."
  END IF
END PROCEDURE
```

3. Pseudocode for Grade Calculation:

```
FUNCTION Calculate_Grade(average)
  IF average >= 90 THEN
    RETURN "A+"
  ELSE IF average >= 80 THEN
    RETURN "A"
  ELSE IF average >= 70 THEN
    RETURN "B+"
  ELSE IF average >= 60 THEN
    RETURN "B"
  ELSE IF average >= 50 THEN
    RETURN "C"
  ELSE IF average >= 40 THEN
    RETURN "D"
  ELSE
    RETURN "F"
  END IF
END FUNCTION
```

4. Pseudocode for Average Calculation:

```
FUNCTION Calculate_Average(marks)
  total = SUM(marks)
  average = total / LENGTH(marks)
  RETURN average
END FUNCTION
```

5. Pseudocode for the Main Display Menu:

```
PROCEDURE Main_Menu
    WHILE TRUE
        PRINT "1. Add Student"
        PRINT "2. Display Records"
        PRINT "3. Search Student"
        PRINT "4. Calculate Average/Grade"
        PRINT "5. Exit"

        INPUT choice

        IF choice == "1" THEN
            CALL Add_Student
        ELSE IF choice == "2" THEN
            CALL Display_Students
        ELSE IF choice == "3" THEN
            CALL Search_Student
        ELSE IF choice == "4" THEN
            CALL Display_Grade
        ELSE IF choice == "5" THEN
            PRINT "Program terminated."
            BREAK
        ELSE
            PRINT "Invalid choice."
        END IF
    END WHILE
END PROCEDURE
```

- Source Code:

```
students = []

def calculate_average(marks):
    total = sum(marks)
    average = total / len(marks)
    return average

def calculate_grade(average):
    if average >= 90:
        return "A+"
    elif average >= 80:
        return "A"
    elif average >= 70:
        return "B+"
    elif average >= 60:
        return "B"
    elif average >= 50:
        return "C"
    elif average >= 40:
        return "D"
    else:
        return "F"

def add_student():
    print("\n--- Add Student ---")

    name = input("Enter student name: ")
    prn = input("Enter PRN: ")
    department = input("Enter department: ")
    semester = int(input("Enter semester: "))

    marks = []

    print("\nEnter marks for 5 subjects:")

    for i in range(5):
        mark = float(input("Enter marks for Subject " + str(i + 1) + ": "))
        marks.append(mark)

    attendance = float(input("\nEnter attendance percentage: "))

    average = calculate_average(marks)
    grade = calculate_grade(average)
```

```

student = {
    "name": name,
    "prn": prn,
    "department": department,
    "semester": semester,
    "marks": marks,
    "attendance": attendance,
    "average": average,
    "grade": grade
}

students.append(student)

print("\nStudent record added successfully!")

def display_students():
    print("\n--- Student Records ---")

    if len(students) == 0:
        print("No student records available.")
        return

    for student in students:
        print("Name      :", student["name"])
        print("PRN       :", student["prn"])
        print("Department :", student["department"])
        print("Semester   :", student["semester"])

        print("Subject Marks:")
        for i in range(5):
            print("  Subject", i + 1, ":", student["marks"][i])

        print("Attendance :", student["attendance"], "%")
        print("Average    :", round(student["average"], 2))
        print("Grade      :", student["grade"])

def search_student():
    print("\n Search Student")

    if len(students) == 0:
        print("No student records available.")
        return

    prn = input("Enter PRN to search: ")

    found = False

```

```

for student in students:
    if student["prn"] == prn:
        print("\nStudent Found!")
        print("Name      :", student["name"])
        print("PRN:", student["prn"])
        print("Department :", student["department"])
        print("Semester  :", student["semester"])
        print("Marks      :", student["marks"])
        print("Attendance :", student["attendance"], "%")
        print("Average   :", round(student["average"], 2))
        print("Grade     :", student["grade"])

        found = True
        break

if found == False:
    print("Student with PRN", prn, "not found.")

def display_grade():
    print("\n Average Marks and Grade")

    if len(students) == 0:
        print("No student records available.")
        return

    prn = input("Enter PRN: ")

    found = False

    for student in students:
        if student["prn"] == prn:

            average = calculate_average(student["marks"])
            grade = calculate_grade(average)

            print("\nStudent Name :", student["name"])
            print("Average Marks:", round(average, 2))
            print("Grade       :", grade)

            found = True
            break

    if found == False:
        print("Student not found.")

while True:

```

```
print(" STUDENT RECORD AND ACADEMIC SYSTEM")

print("1. Add Student")
print("2. Display Student Records")
print("3. Search Student")
print("4. Calculate Average and Grade")
print("5. Exit")

choice = input("\nEnter your choice: ")

if choice == "1":
    add_student()

elif choice == "2":
    display_students()

elif choice == "3":
    search_student()

elif choice == "4":
    display_grade()

elif choice == "5":
    print("\nThank you for using the Student Record System.")
    print("Program terminated.")
    break

else:
    print("\nInvalid choice. Please enter a number from 1 to 5.")
```

- Testing and Screenshots:

1. Test Case 1 (Add Student):

```
STUDENT RECORD AND ACADEMIC SYSTEM
1. Add Student
2. Display Student Records
3. Search Student
4. Calculate Average and Grade
5. Exit

Enter your choice: 1

--- Add Student ---
Enter student name: Jack
Enter PRN: 285
Enter department: AIML
Enter semester: 1

Enter marks for 5 subjects:
Enter marks for Subject 1: 86
Enter marks for Subject 2: 75
Enter marks for Subject 3: 77
Enter marks for Subject 4: 89
Enter marks for Subject 5: 84

Enter attendance percentage: 86

Student record added successfully!
```

2. Test Case 2 (Boundary Testing):

- i. Test For A+ grade:

```
--- Add Student ---
Enter student name: Jack
Enter PRN: 285
Enter department: AIML
Enter semester: 1

Enter marks for 5 subjects:
Enter marks for Subject 1: 90
Enter marks for Subject 2: 90
Enter marks for Subject 3: 90
Enter marks for Subject 4: 90
Enter marks for Subject 5: 90

Enter attendance percentage: 86

Student record added successfully!
STUDENT RECORD AND ACADEMIC SYSTEM
1. Add Student
2. Display Student Records
3. Search Student
4. Calculate Average and Grade
5. Exit

Enter your choice: 4

Average Marks and Grade
Enter PRN: 285

Student Name : Jack
Average Marks: 90.0
Grade       : A+
```

ii. Test for A grade:

```
--- Add Student ---
Enter student name: Jack
Enter PRN: 285
Enter department: AIML
Enter semester: 1

Enter marks for 5 subjects:
Enter marks for Subject 1: 89
Enter marks for Subject 2: 89
Enter marks for Subject 3: 89
Enter marks for Subject 4: 89
Enter marks for Subject 5: 89

Enter attendance percentage: 86

Student record added successfully!
  STUDENT RECORD AND ACADEMIC SYSTEM
1. Add Student
2. Display Student Records
3. Search Student
4. Calculate Average and Grade
5. Exit

Enter your choice: 4

  Average Marks and Grade
Enter PRN: 285

Student Name : Jack
Average Marks: 89.0
Grade       : A
```

3. Test Case 3 (Error/Logical Testing):

```
STUDENT RECORD AND ACADEMIC SYSTEM
1. Add Student
2. Display Student Records
3. Search Student
4. Calculate Average and Grade
5. Exit

Enter your choice: 9

Invalid choice. Please enter a number from 1 to 5.
```

- Unit Reflection Report:

Throughout this project, I learned that effective software development requires careful planning before any code is written. By breaking down the problem and creating algorithms, flowcharts, and pseudocode first, I significantly improved my computational thinking and logical reasoning skills. This structured methodology made the actual Python implementation much easier and less prone to errors. Additionally, I gained valuable hands-on experience using loops, conditionals, and dictionaries in Python, as well as learning how to systematically test my program for boundary and logical errors.